DNSFilter

Reference · 2026

Command-line interface for the DNSFilter API

dnsfcli Reference Guide

Complete command reference, tested examples, CSV bulk-import workflows, and output flag usage for every endpoint in the DNSFilter API.

TOOL VERSION	UPDATED	AUDIENCE
0.1.0	2026-07-13	Developers & Administrators

Overview

dnsfcli is a Python command-line tool that wraps the entire DNSFilter REST API. Every endpoint, every write field, and every output mode is available directly from the terminal — no browser required.

> **COMMAND STRUCTURE**
>
> All commands follow a single three-part pattern: `dnsfcli.py [endpoint] [function] [--param value …]`. The endpoint is the resource group (e.g. `networks`, `policies`), the function is the operation (e.g. `list`, `create`, `update`), and flags supply the values.

The tool routes to 242 operations across 36 resource groups, all defined in a single endpoint registry. Mistyped endpoint or function names exit non-zero with a “did you mean” suggestion.

Installation & Requirements

1 Install from the repository

pip installs straight from GitHub — re-run the same command to upgrade. To work from source, the tool lives in the `dnsfcli/` directory of the public `DNSFilter/support` repository and uses a standard `src`-layout Python package.

2 Install dependencies

Python 3.11 or later is required. Install with pip from the project root.

SHELL / INSTALL

```
pip install "git+https://github.com/DNSFilter/support.git#subdirectory=dnsfcli"
# or, from a local clone:
git clone https://github.com/DNSFilter/support
cd support/dnsfcli
```

```
pip install -e .
# installs: typer, click, httpx, keyring, rich
```

3 Run directly or as a module

Both invocation styles are equivalent. The entry-point script `dnsfcli.py` in the project root is the recommended way to run it during development.

SHELL / INVOCATION OPTIONS

```
python dnsfcli.py [endpoint] [function] [flags]
python -m dnsfcli  [endpoint] [function] [flags]
dnsfcli            [endpoint] [function] [flags]   # after pip install
```

Authentication

The tool stores credentials in the OS keychain (macOS Keychain, Windows Credential Manager, Linux Secret Service). Credentials are read automatically on every command — no manual token passing required once set up.

1 Store your API token

Run `auth setup` and enter your DNSFilter JWT or API key when prompted. Optionally set a default org ID so it pre-fills templates and org-scoped calls.

SHELL / AUTH SETUP

```
python dnsfcli.py auth setup
# DNSFilter API key: ••••••••••••••

python dnsfcli.py auth setup --org-id 802315
```

2 Verify the credentials work

`auth verify` makes a live call to `/v1/organizations` and confirms the token is valid before you run anything else.

SHELL / AUTH VERIFY

```
python dnsfcli.py auth verify
✓ Credentials are valid.

python dnsfcli.py auth show
  api_key    eyJhbG...k3Qw
  org_id     802315
  base_url   https://api.dnsfilter.com
```

ENVIRONMENT VARIABLE OVERRIDE

Set `DNSF_API_KEY` to override the keychain token for a single session. Pass `--api-key <TOKEN>` on any command to override for a single call.

Global Flags

These flags work on every command and can be placed anywhere in the argument list — before the endpoint, between endpoint and function, or after all other flags.

Output & format	
`--raw, -r`	Print the raw JSON response instead of the formatted table.
`--json / --jsonl`	JSON (or one object per line) on stdout. Automatic when output is piped.
`--output FMT`	Unified format switch: table, json, jsonl, raw, csv, or none.
`--columns a,b,c`	Limit table/CSV output to the named columns.
`--preset NAME`	Apply a named column preset from config (column_presets.NAME).
`--format TMPL`	Render each row through a template — both "{name} ({id})" and "{{.name}}" styles work.
`--format-preset N`	Apply a named format template from config (format_presets.NAME).
`--bundle NAME`	Apply a named command bundle from config (columns + filter + sort + format).
`--pick FIELD`	Print a single field, one value per line — ideal for piping.
`--jq PATH`	Extract a dotted path (data.0.name) from the response before output.

`--to-csv FILE`	Write the response to FILE as CSV (use - for stdout).
`--to-json FILE`	Write the response to FILE as JSON.
`--to-markdown FILE`	Write the response to FILE as a Markdown table (use - for stdout).
`--tee FILE`	Save a copy of everything printed to FILE.
`--quiet, -q`	Suppress status chatter; result data still prints.
`--no-color`	Disable ANSI colour output (NO_COLOR env also honoured).

Filtering, sorting & aggregation

`--filter EXPR`	Keep matching rows: field=value, field!=value, field~substr, field>N, >=, <, <=.
`--filter-mode or`	OR logic across multiple --filter flags (default: AND).
`--grep REGEX`	Keep rows where any field matches the regex.
`--unique FIELD`	Drop rows with a duplicate value in FIELD.
`--sort FIELD`	Sort by field; prefix with - for descending (--sort -created_at).
`--limit N / --last N`	Return at most N results / the last N results.
`--count`	Print the result count only.
`--count-by FIELD`	Frequency table of FIELD values.
`--group-by FIELD`	Group results by FIELD value.
`--sum / --avg / --min / --max F`	Aggregate a numeric field and print one value.
`--select a,b / --exclude a,b`	Keep only (or remove) the named fields from every item.
`--not-null F / --is-null F`	Keep rows where FIELD is (or is not) null.

Pagination

`--all`	Fetch every page of paginated results.
`--page N / --page-size N`	Request a specific page / page size.
`--max-pages N`	Cap the number of pages fetched by --all.
`--paginate-until E`	Stop --all pagination once any item matches filter expression E.

Batch & CSV input

`--from-csv FILE`	Read input rows from FILE — one API call per row (use - for stdin).
`--template`	Print a blank CSV import template for this command and exit. No auth needed.
`--skip-rows N / --max-rows N`	Resume an interrupted batch / cap rows processed.
`--batch-report FILE`	Write a JSON run summary (per-row outcomes, counts) to FILE.
`--on-error MODE`	stop or continue when a batch row fails.
`--errors-to-csv FILE`	Write failed rows to FILE for later retry (--retry-errors-csv).
`--confirm-each`	Prompt before each batch row.
`--validate-only`	Validate the CSV and exit without calling the API.
`--dry-run / --plan`	Show what would be sent without making any API calls.
`--yes, -y`	Skip confirmation prompts.

Transformation	
`--add-field K=V`	Inject a static field into every result item.
`--transform F=EXPR`	Compute a new field from a restricted expression (ratio=blocked/total).
`--map FIELD=OP`	Transform a field value: upper, lower, strip, title, truncate:N.
`--join EP:LK=RK`	Client-side join: fetch endpoint EP, match local key LK to remote key RK.
`--rename A=B`	Rename field A to B in the output.
`--flatten`	Flatten nested objects into dotted keys.
`--strip-nulls`	Remove null-valued keys from every item.

Multi-organization	
`--each-org`	Run the command once per organization on the account.
`--org-csv FILE`	Load the --each-org organization list from a CSV file.
`--org-filter REGEX`	Restrict --each-org to organizations whose name matches.
`--max-orgs N`	Cap the number of organizations processed.
`--parallel-orgs`	Process organizations concurrently (--org-concurrency N).

Watch, monitor & CI	
`--watch N`	Re-run the command every N seconds (Ctrl-C to stop).
`--watch-until EXPR`	Stop watching once any result matches the filter expression.
`--watch-diff`	Show only what changed between watch ticks.
`--alert EXPR`	Ring the terminal bell and print a banner when a result matches.
`--fail-on-empty`	Exit 1 when the result list is empty (messages go to stderr).
`--fail-on-pattern E`	Exit 1 when any result row matches filter expression E.
`--exec CMD`	Run a shell command per result row with {field} substitution (values are shell-quoted).

Connection & authentication	
`--api-key TOKEN`	Override the keychain token for this call only (prefer DNSF_API_KEY).
`--org-id ID`	Override the stored organization ID for this call only.
`--profile NAME`	Use a named credential profile.
`--timeout N / --connect-timeout N`	Read / connect timeouts in seconds.
`--rate N`	Client-side request-per-second cap.
`--retry N`	Retry attempts for failed batch rows.
`--header K=V`	Add a custom request header (scrubbed from history logs).
`--proxy URL`	Route requests through a proxy (scrubbed from history logs).
`--env-file FILE`	Load KEY=VALUE pairs into the environment before running.
`--cache-ttl N`	Serve identical GETs from a local cache for N seconds.

Discovery

Two built-in commands let you explore the full endpoint catalogue without leaving the terminal.

SHELL / LIST ALL ENDPOINTS

```
python dnsfcli.py endpoints
# Endpoint                Functions
#
-------------------------------------------------------------------------------
# agent-local-users       bulk-delete, delete, list, show, update ⋯
# api-keys                create, delete, list, revoke, show
# networks                bulk-create, counts, create, delete, list ⋯
# policies                add-blacklist-domain, create, delete, list ⋯
# traffic-reports         qps, query-logs, top-domains, total-requests ⋯
# ⋯ (36 groups total)
```

SHELL / FUNCTIONS FOR ONE ENDPOINT

```
python dnsfcli.py endpoints policies
# add-allowed-application   add-blacklist-category
# add-blacklist-domain      add-whitelist-domain
# application               bulk-add-allowlist
# create                    delete
# list                      list-all
# permissive-mode           remove-blacklist-domain
# set-permissive-mode       show
# update
```

Core Resource Operations

Every major resource follows the same five-function pattern. The examples below use **policies** but the same functions apply to networks, organizations, block-pages, ip-addresses, mac-addresses, and scheduled-policies.

SHELL / LIST

```
python dnsfcli.py policies list

#
┌──────────┬──────────┬──────────────────────┬──────────────────┐
# | id       | type     | attributes           | relationships    |
#
├──────────┼──────────┼──────────────────────┼──────────────────┤
# | 331207   | policies | name=big bada        | organization=…   |
# |          |          | blocklist,           |                  |
# |          |          | organization_id=…    |                  |
# | 285109   | policies | name=Block Adult     | organization=…   |
# |          |          | Content, …           |                  |
#
└──────────┴──────────┴──────────────────────┴──────────────────┘

python dnsfcli.py policies list --raw | jq '.data[].attributes.name'
```

SHELL / SHOW A SINGLE RESOURCE

```
python dnsfcli.py policies show --id 285109

#  ╭──────────────────────────────────────── policies show
──────────────────────────────────────────╮
# | id                          285109                    |
# | attributes.name             Block Adult Content       |
# | attributes.organization_id  802315                    |
# | attributes.blacklist_categories 56, 69, 2, 38         |
# | attributes.google_safesearch    no                    |
# | attributes.youtube_restricted   yes                   |
# | attributes.allow_unknown_domains yes                  |
#
 ╰──────────────────────────────────────────────────────────────────────────────╯
```

SHELL / CREATE

```
python dnsfcli.py policies create  --name "Guest WiFi"  --organization_id 802315  --allow_unknown_domains true  --youtube_restricted true  --youtube_restricted_level strict  --google_safesearch true  --bing_safe_search true
```

SHELL / UPDATE

```
python dnsfcli.py policies update --id 285109  --name "Block Adult Content v2"  --interstitial true
```

SHELL / DELETE

```
python dnsfcli.py policies delete --id 285109
```

Networks

POLICY ASSIGNMENT USES AN ARRAY

Networks use `policy_ids` (a JSON array), not a single `policy_id`. Pass it as a JSON array string: `--policy_ids '["285109","331207"]'`.

SHELL / NETWORKS — KEY OPERATIONS

```
# List with org filter
python dnsfcli.py networks list --organization_id 802315

# Create with policy assignment
python dnsfcli.py networks create  --name "Branch Office"  --organization_id 802315  --policy_ids '["285109"]'  --physical_address "123 Main St, Denver CO"

# Network counts
python dnsfcli.py networks counts

# Geographic data
python dnsfcli.py networks geo

# LAN IP management
```

```
python dnsfcli.py networks lan-ips --id 736401
python dnsfcli.py networks lan-ip-update --id 736401 --lan_ip_id 42  --name "Reception Desk"

# Subnets (use 'from' and 'to', not cidr)
python dnsfcli.py networks subnets-create --id 736401  --name "Sales Floor" --from "10.0.1.0" --to "10.0.1.255"  --policy_id 285109

# Secret key rotation
python dnsfcli.py networks secret-key-create --id 736401
```

Policy Domain & Category Management

Individual domains and categories are added or removed from a policy with targeted action functions. Application filtering uses the application name string, not an ID.

SHELL / POLICY DOMAIN & CATEGORY ACTIONS

```
# Block a domain
python dnsfcli.py policies add-blacklist-domain  --id 285109 --domain "malware.example.com"  --note "Flagged by threat intel"

# Allow a domain
python dnsfcli.py policies add-whitelist-domain  --id 285109 --domain "internal.corp.com"

# Block a category (Adult Content = 2)
python dnsfcli.py policies add-blacklist-category  --id 285109 --category_id 2

# Block an application by name
python dnsfcli.py policies add-blocked-application  --id 285109 --name "TikTok"

# Remove a domain from blocklist
python dnsfcli.py policies remove-blacklist-domain  --id 285109 --domain "malware.example.com"

# Bulk add domains to multiple policies at once
```

```
python dnsfcli.py policies bulk-add-blocklist  --policy_ids '["285109","331207"]'  --domains '["evil.com","malware.net"]'
```

APPLICATION NAMES ARE CASE-SENSITIVE

The `add-blocked-application` and `add-allowed-application` functions take the application `name` field exactly as it appears in `python dnsfcli.py applications list`. Mismatched casing returns a 404.

Organizations & Users

SHELL / ORGANIZATIONS

```
# List all organizations
python dnsfcli.py organizations list

# Show one organization
python dnsfcli.py organizations show --id 802315

# Create an organization
python dnsfcli.py organizations create  --name "New Client Corp"  --billing_contact_email "billing@newclient.com"  --sku "professional"

# Organization settings
python dnsfcli.py organizations settings

# Promote to MSP
python dnsfcli.py organizations promote-to-msp
```

SHELL / ORGANIZATION USERS

```
# List users in an org
python dnsfcli.py organizations users-list --organization_id 802315

# Add a user
python dnsfcli.py organizations users-create  --organization_id 802315  --email
```

```
"newuser@company.com"  --first_name "Jane" --last_name "Smith"  --role "administrator"

# Update a user's role
python dnsfcli.py organizations users-update  --organization_id 802315 --id 42618  --role "read_only"

# Remove a user
python dnsfcli.py organizations users-delete  --organization_id 802315 --id 42618
```

IP & MAC Addresses

FIELD NAME CORRECTION

The API field is `address`, not `ip_address` or `mac_address`. Both IP and MAC address endpoints use the same `address` parameter name.

SHELL / IP & MAC ADDRESSES

```
# Add a static IP to a network
python dnsfcli.py ip-addresses create  --address "203.0.113.5"  --organization_id 802315  --network_id 736401

# Add a MAC address
python dnsfcli.py mac-addresses create  --organization_id 802315  --address "AA:BB:CC:DD:EE:FF"  --filter_value "Reception Printer"  --policy_id 285109

# Show my current IP
python dnsfcli.py ip-addresses myip
```

Roaming Agents (User Agents)

SHELL / USER AGENTS

```
# List all agents with filters
python dnsfcli.py user-agents list  --organization_id 802315  --network_id 736401

# Agent counts
python dnsfcli.py user-agents counts

# Update an agent (note: display name field is 'friendly_name')
python dnsfcli.py user-agents update  --id "9b2f6c04-5a1e-4d37-8c2a-f0e1d2c3b4a5"  --
friendly_name "Finance-Laptop"  --policy_id 285109  --tags '["managed","finance"]'

# Bulk update multiple agents at once
python dnsfcli.py user-agent-bulk-updates create  --ids '["9b2f6c04-5a1e-4d37-8c2a-
f0e1d2c3b4a5"]'  --policy_id 285109

# CSV export of all agents
python dnsfcli.py user-agents csv > agents.csv

# Uninstall PIN
python dnsfcli.py user-agents uninstall-pin
```

Traffic Reports

All 51 traffic report endpoints are GET-only and require `--start_date` and `--end_date` in `YYYY-MM-DD` format. Results can be scoped to an org or network with optional filters.

SHELL / TRAFFIC REPORTS

```
# Total requests in January
python dnsfcli.py traffic-reports total-requests  --start_date 2025-01-01 --end_date
2025-01-31

# Top blocked domains last 30 days
python dnsfcli.py traffic-reports top-domains  --start_date 2025-01-01 --end_date
2025-01-31  --organization_id 802315

# Threat summary
python dnsfcli.py traffic-reports total-threats  --start_date 2025-01-01 --end_date
2025-01-31
```

```
# Query logs (export raw DNS events)
python dnsfcli.py traffic-reports query-logs  --start_date 2025-01-01 --end_date
2025-01-31  --csv query-logs-jan.csv

# QPS for active agents
python dnsfcli.py traffic-reports qps-active-agents  --start_date 2025-01-01 --end_date
2025-01-31
```

SOME REPORT ENDPOINTS ENFORCE A MAXIMUM TIME WINDOW

`total-client-stats` and a few QPS endpoints accept only short time ranges (roughly 20 minutes). Use them for real-time monitoring, not historical analysis.

Domain Lookups & Classification

SHELL / DOMAIN LOOKUPS

```
# Bulk classify multiple domains
python dnsfcli.py domains bulk-lookup  --domains
"google.com,facebook.com,malware.example"

# Look up a domain for a specific user session
python dnsfcli.py domains user-lookup --domain "google.com"

# Suggest a domain for threat review
python dnsfcli.py domains suggest-threat  --domain "suspicious.example.com"  --reason
"Flagged in phishing email campaign"
```

API Key Management

SHELL / API KEYS

```
# List all API keys
python dnsfcli.py api-keys list

# Create a new key (expiry must be within 1 year)
python dnsfcli.py api-keys create  --name "CI Pipeline Key"  --expiry "2027-05-31"

# Revoke a key immediately
python dnsfcli.py api-keys revoke --id 99

# Delete a key permanently
python dnsfcli.py api-keys delete --id 99
```

Scheduled Reports & Policies

SHELL / SCHEDULED REPORTS

```
# Create a weekly threat report
python dnsfcli.py scheduled-reports create  --organization_id 802315  --frequency
"weekly"  --day_of_week "1"  --include_threat_summary true  --
include_content_category_summary true  --send_to_dashboard_users true

# Preview a report immediately
python dnsfcli.py scheduled-reports preview-create  --organization_id 802315  --
include_threat_summary true
```

SHELL / SCHEDULED POLICIES

```
# Create a time-based policy schedule
python dnsfcli.py scheduled-policies create  --name "School Hours"  --organization_id
802315  --policy_ids '["285109"]'  --timezone "America/Denver"
```

v2 Endpoints

The v2 API surface covers Cyber Sight, CSV exports, UI settings, and VPN dictionary data. Commands follow the same structure using the `v2-` endpoint prefix.

SHELL / V2 ENDPOINTS

```
# UI settings for current user
python dnsfcli.py v2-current-user ui-settings
python dnsfcli.py v2-current-user ui-settings-update   --theme_mode "dark"

# Cyber Sight activity types reference
python dnsfcli.py v2-dictionary cyber-sight-activity-types

# VPN settings state types
python dnsfcli.py v2-dictionary vpn-settings-state-types

# Cyber Sight CSV export
python dnsfcli.py v2-cyber-sight csv-export   --organization_ids '["802315"]'   --threats_only
true   --start_at "2025-01-01T00:00:00Z"   --end_at "2025-01-31T23:59:59Z"

# Agent local user counts (v2)
python dnsfcli.py v2-agent-local-users counts

# Networks CSV export (v2)
python dnsfcli.py v2-networks csv-export   --organization_ids '["802315"]'
```

Output Modes

By default every command renders a human-readable rich table. Three flags change the output format.

SHELL / OUTPUT — TABLE (DEFAULT)

```
python dnsfcli.py policies show --id 285109
# ╭──────────────────────────────────────── policies show
─────────────────────────────────────────╮
```

```
# │ attributes.name              Block Adult Content     │
# │ attributes.organization_id   802315                  │
# │ attributes.youtube_restricted  yes                   │
# │ attributes.google_safesearch   no                    │
#
```

SHELL / OUTPUT — RAW JSON (--RAW)

```
python dnsfcli.py policies show --id 285109 --raw
# {
#   "id": 285109,
#   "type": "policies",
#   "attributes": {
#     "name": "Block Adult Content",
#     "organization_id": 802315,
#     ...
#   }
# }

# Pipe directly into jq
python dnsfcli.py policies list --raw | jq '.[].attributes.name'
```

SHELL / OUTPUT — CSV FILE (--CSV)

```
# Save list results to a CSV file
python dnsfcli.py policies list --csv policies.csv
# ✓ Wrote 8 rows to policies.csv

# Traffic report to CSV for Excel/Sheets
python dnsfcli.py traffic-reports total-requests   --start_date 2025-01-01 --end_date
2025-01-31   --csv jan-traffic.csv

# Combine --raw and --csv: raw JSON to screen, CSV saved simultaneously
python dnsfcli.py networks list --raw --csv networks-backup.csv
```

Bulk CSV Import (--from-csv)

Any write operation can accept a CSV file as its input source. The tool validates every row before making a single API call, then processes them sequentially with per-row status output.

1 Generate a blank template

The `--template` flag prints a correctly structured CSV with comment lines documenting required and optional fields. If a default org ID is stored in the keychain, it is pre-filled in the example row.

SHELL / GENERATE TEMPLATE

```
python dnsfcli.py policies create --template

# Template : dnsfcli policies create
# Required : name (string), organization_id (integer)
# Optional : allow_unknown_domains (boolean), google_safesearch (boolean) ⋯
name,organization_id,allow_unknown_domains,google_safesearch, ⋯
example text,802315,,,⋯

# Save directly to file
python dnsfcli.py policies create --template > policies-template.csv
```

2 Fill in the template

Open the CSV in Excel, Google Sheets, or any editor. Fill in one row per resource. The comment lines (# prefix) can be left in — they are skipped by the importer. Arrays use JSON syntax: `["item1","item2"]`.

CSV / FILLED TEMPLATE EXAMPLE

```
# Template : dnsfcli policies create
# Required : name (string), organization_id (integer)
name,organization_id,google_safesearch,youtube_restricted,allow_unknown_domains
Guest WiFi,802315,true,true,true
```

```
Employee Standard,802315,true,false,false
Kiosk Restrictive,802315,true,true,false
```

3 Import with --from-csv

The tool validates all rows first. Any errors are shown with row numbers and the field that failed — and no API calls are made until the entire file is clean.

SHELL / IMPORT

```
python dnsfcli.py policies create --from-csv policies-template.csv

# Processing 3 rows from CSV…
#   Row 1: ✓ (id: 1501234)
#   Row 2: ✓ (id: 1501235)
#   Row 3: ✓ (id: 1501236)
#
# Done: 3/3 rows succeeded
```

PATH PARAMS CAN COME FROM EITHER THE CLI OR THE CSV

Supply path parameters (like `--id`) on the command line to apply the same value to every row. Or include an `id` column in the CSV to target a different resource per row — useful for bulk updates.

SHELL / MIXED CLI + CSV — BULK DOMAIN BLOCK

```
# Block a list of domains against a single policy
# CSV: domain,note
# evil.com,Phishing campaign
# malware.net,C2 infrastructure

python dnsfcli.py policies add-blacklist-domain   --id 285109   --from-csv threats.csv

# Processing 2 rows from CSV…
#   Row 1: ✓
```

```
#   Row 2: ✓
# Done: 2/2 rows succeeded
```

Validation & Error Handling

Errors are always surfaced cleanly — no Python tracebacks are shown to the user. Three categories of errors may occur.

SHELL / ERROR — MISSING PATH PARAMETER

```
python dnsfcli.py policies show
# Error: Required path parameter --id was not provided.
# Path template: /v1/policies/{id}
```

SHELL / ERROR — CSV STRUCTURAL PROBLEM

```
python dnsfcli.py policies create --from-csv bad.csv
# Error: CSV validation failed for bad.csv:
#   Missing required column(s): name
#     Required columns : name (string), organization_id (integer)
#     Optional columns : allow_unknown_domains (boolean) ⋯
#     Run with --template to generate a blank example CSV
#
#   No API calls were made
```

SHELL / ERROR — CSV DATA PROBLEM

```
python dnsfcli.py networks create --from-csv networks.csv
# Error: CSV validation failed for networks.csv:
#   Row 3: 'organization_id' -- expected an integer, got 'acme-corp'
#   Row 5: 'name' is required but empty
#
#   2 error(s) found across 2 row(s) -- no API calls were made
```

SHELL / ERROR — API ERROR

```
python dnsfcli.py policies delete --id 99999
# Error: HTTP 404: Unable to find the object that you requested.
#   error: Unable to find the object that you requested.
```

Retry & Backoff Behaviour

The HTTP client handles transient failures automatically so scripts do not need retry logic of their own.

Condition	Behaviour
HTTP 429 (rate limited)	Reads `Retry-After` header, sleeps that many seconds, retries indefinitely until the server responds with non-429.
Connection error / timeout	Exponential backoff from 1 s, capped at 60 s, up to 6 attempts before raising.
HTTP 5xx server error	Raised immediately as `APIError` — not retried. Operator-side issues are not transient.
HTTP 4xx (except 429)	Raised immediately with the error message from the response body.

RATE LIMIT FOR THE DNSFILTER API

The API enforces 2,000 requests per 300 seconds per organization. Large `--from-csv` imports against a heavily used account may hit this. The client handles the 429 automatically; no special flag is needed.

Complete Write Field Reference

Every write endpoint with its full parameter set. **Bold** fields are required.

Endpoint / Function	Required	Optional
agent-local-users update	id	friendly_name, policy_id, scheduled_policy_id, block_page_id
	ids (array)	exclude_ids (array)

Endpoint / Function	Required	Optional
agent-local-users bulk-delete		
api-keys create	name	expiry (YYYY-MM-DD, max 1 year)
billing update-address	organization_id (path)	first_name, last_name, email, company, phone, line1, line2, line3, city, state, state_code, zip, country
block-pages create / update	name	organization_id, block_org_name, block_email_addr, block_logo_uuid
current-user update	—	first_name, last_name, phone
ip-addresses create / update	address, organization_id, network_id	dynamic_hostname
mac-addresses create / update	organization_id	address, filter_value, policy_id, scheduled_policy_id, block_page_id
networks create / update	name, organization_id	block_page_id, policy_ids (array), external_id, is_legacy_vpn_active, physical_address, local_domains (array), local_resolvers (array)
networks subnets-create / update	id (network), name, from (start IP), to (end IP)	policy_id, scheduled_policy_id, block_page_id
organizations create / update	name	billing_contact_name, billing_contact_phone, billing_contact_email, address, show_pii_rc_hostnames, unique_id, sku, quantity, gdpr, privacy_mode, enable_cybersight
organizations users-create / update	organization_id (path)	email, first_name, last_name, phone, role, organization_permission_ids (array), is_include_only_list
policies create / update	name, organization_id	allow_unknown_domains, google_safesearch, bing_safe_search, duck_duck_go_safe_search, ecosia_safesearch, yandex_safe_search, youtube_restricted, youtube_restricted_level, interstitial, allow_list_only, is_global_policy, policy_ip_id, whitelist_domains (array), blacklist_domains (array), blacklist_categories (array), allow_applications (array), block_applications (array), append_domains, include_relationships
policies add/ remove domain	id, domain	note, include_relationships
policies add/ remove category	id, category_id	include_relationships
policies add/ remove application	id, name (app name string)	include_relationships
	policy_ids (array), domains (array)	—

Endpoint / Function	Required	Optional
policies bulk-add/remove domain lists		
scheduled-policies create / update	name	organization_id, policy_ids (array), timezone
scheduled-reports create / update	—	organization_id, frequency, day_of_week, include_threat_summary, include_content_category_summary, content_categories_show_count, send_to_dashboard_users, scheduled_report_recipients (array), selected_sub_orgs (array)
user-agent-bulk-updates create	—	ids (array), exclude_ids (array), network_id, policy_id, scheduled_policy_id, block_page_id, friendly_name, tags (array), release_channels (array)
user-agents update	id	friendly_name, status, network_id, policy_id, scheduled_policy_id, block_page_id, tags (array)
users change-password	new_password	—
v2-current-user ui-settings-update	—	disable_license_warnings, user_uuid, theme_mode

Quick Start

01 Run `python dnsfcli.py auth setup` and store your API token and org ID.

02 Run `python dnsfcli.py endpoints` to see all 40 resource groups, then `python dnsfcli.py [endpoint] --template` to get a pre-filled CSV for any write operation.

03 Run `python dnsfcli.py policies list --csv policies-backup.csv` to export a baseline snapshot before making bulk changes.

From the team at

DNSFilter